Academic Project Planning and Automated Architecture Design Tool

Guide Name:- Swathi J.N
Student Name:- Akhya Sinha
Reg.No :- 23BKT0045
Course Code :- BCSE301P [Software Engineering Lab]

Problem Statement

In academic environments, students often face difficulty in planning software projects and deciding an appropriate system architecture. This leads to poor time management and improper system design. This project aims to develop an automated tool that assists students in generating a basic project plan and a high-level architectural design based on simple project inputs.

Base paper - AI-Driven Decision Support Systems for Software Architecture

Abstract

Software project planning and architectural design are critical early-stage activities in the software development life cycle, yet they are often performed manually, leading to inconsistencies, design flaws, and inefficient resource utilization especially in academic and small scale projects. Recent research highlights the growing role of automation, decision support systems, and AI-assisted techniques in improving project planning and architectural decision making. Studies on automated planning, architecture recommendation systems, and generative AI demonstrate that rule-based and intelligent tools can effectively support developers by reducing cognitive load and standardizing design outcomes. Building on these foundations, this work proposes an automated Academic Project Planning and High Level Architecture Design Tool that generates phase-wise project plans and recommends suitable architectural styles based on basic project parameters such as project type, team size, duration, and technology. Unlike complex AI-driven systems, the proposed tool adopts a lightweight, rule-based approach, making it easy to develop, understand, and deploy in educational environments. The tool aligns with existing research on architectural decision support while focusing on simplicity, transparency, and academic usability. This approach demonstrates that even minimal automation can significantly enhance planning accuracy and architectural clarity for novice developers and students, while serving as a foundation for future intelligent extensions.

Process

1. **Requirement Analysis**
   We identify and document the inputs, outputs, and functional behavior required for automating project planning and high-level architecture generation.
2. **System Design**
   We design the layered system architecture and define rule-based logic for generating project plans and architectural recommendations.
3. **Database / Data Structure Design**
   We design simple data structures to store project parameters and generated planning results.
4. **Implementation**
   We develop Python functions to generate phase-wise project plans and automatically suggest suitable system architectures.
5. **User Interface Development**
   We develop a basic console or Flask-based web interface to collect user inputs and display generated outputs.

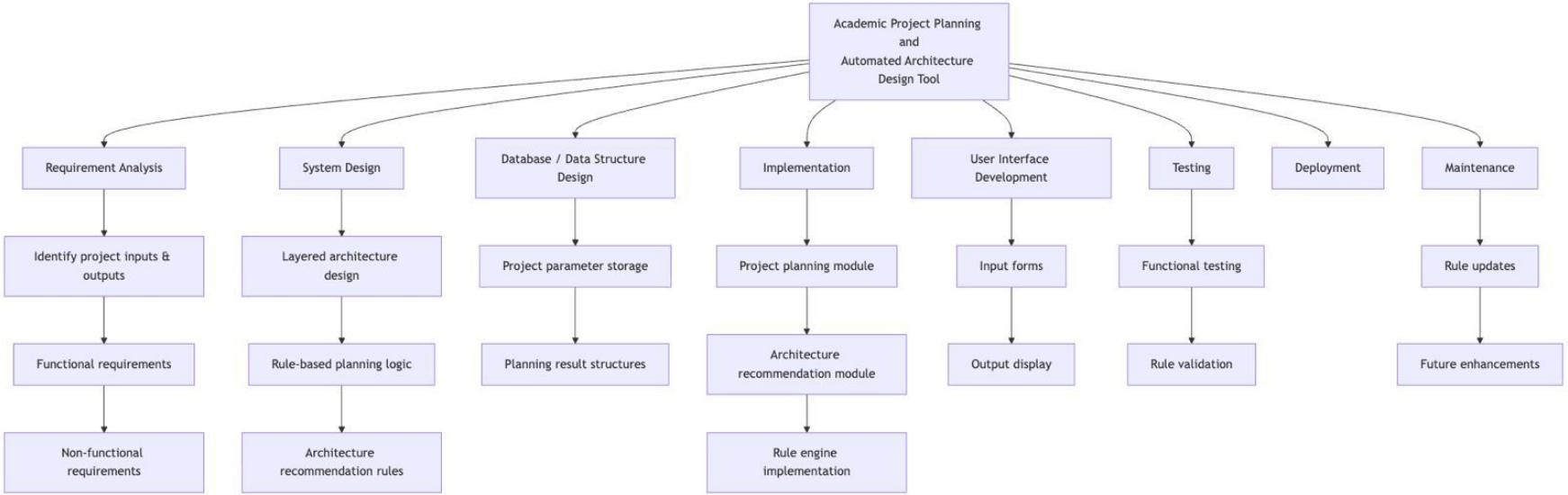
Objectives

1. To automate the planning of academic software projects using basic project inputs.
2. To generate a clear, phase-wise project schedule for effective time management.
3. To automatically suggest an appropriate high-level software architecture.
4. To simplify early-stage project planning and design for students using a rule-based approach.

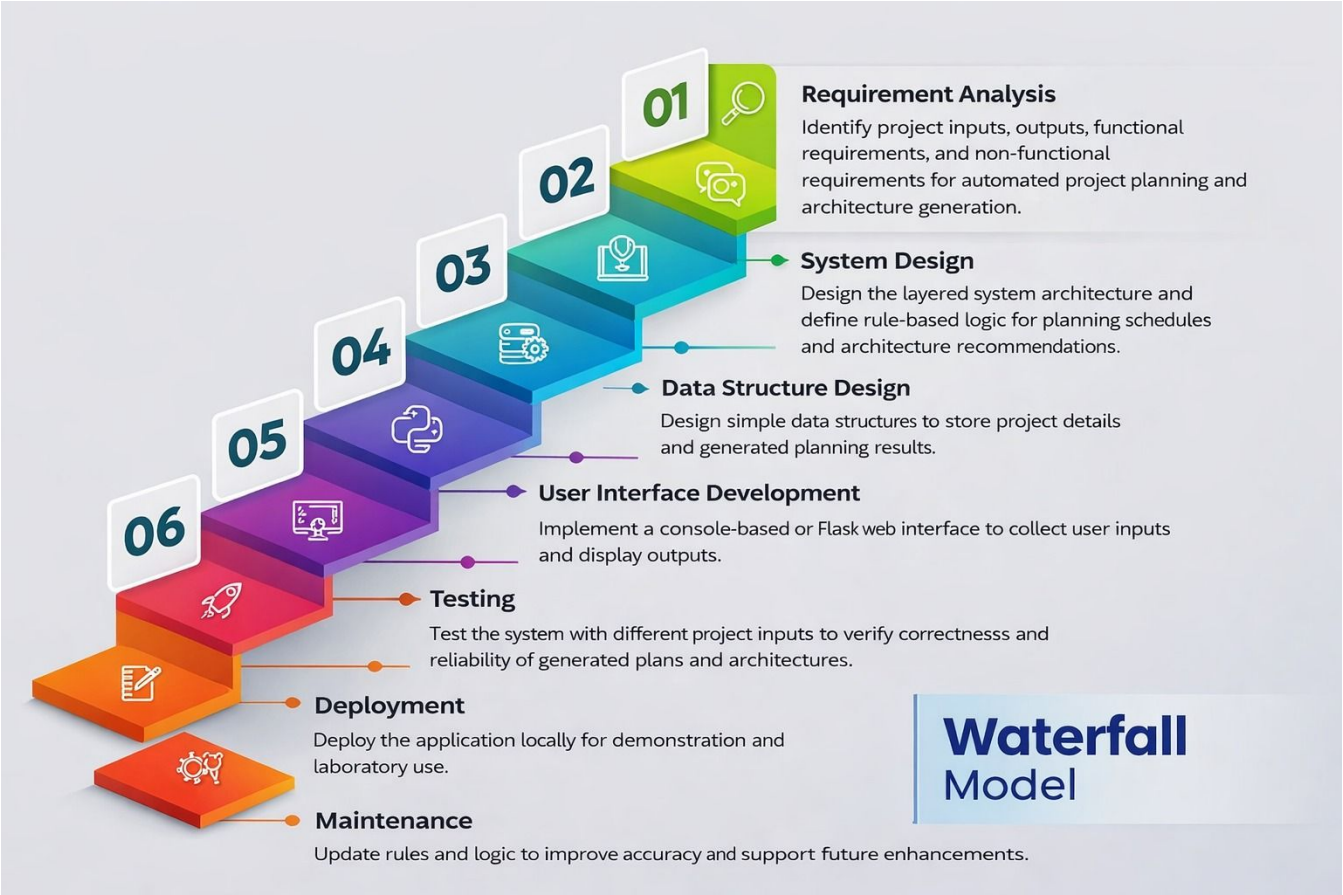
Justification of Project Necessity

1. Many students face difficulty in planning software projects due to limited practical exposure to software engineering methodologies.
2. Manual project planning often leads to improper time allocation and incomplete development phases.
3. Students frequently struggle to select an appropriate high-level architecture for their projects.
4. Lack of structured guidance results in inconsistent and poorly designed academic projects.
5. Automated planning helps reduce human errors and improves project clarity at an early stage.
6. Existing industry-level project management and design tools are too complex for academic use.
7. The proposed tool provides a simple, rule-based solution that supports learning and improves project quality in software engineering laboratories

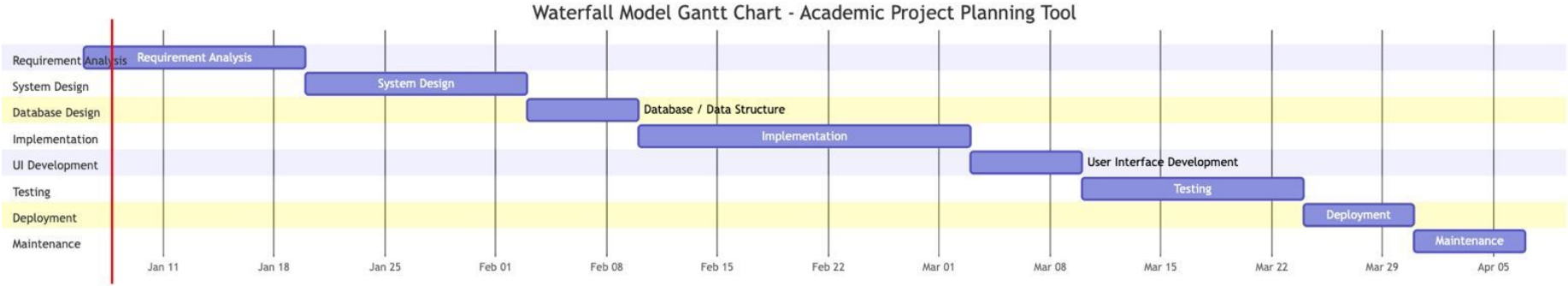
WORK BREAKDOWN STRUCTURE-FLOWCHART



The Work Breakdown Structure (WBS) is a hierarchical decomposition of the project into smaller, manageable components. For the Academic Project Planning and Automated Architecture Design Tool, the WBS breaks the project into major phases such as requirement analysis, system design, implementation, testing, deployment, and maintenance.

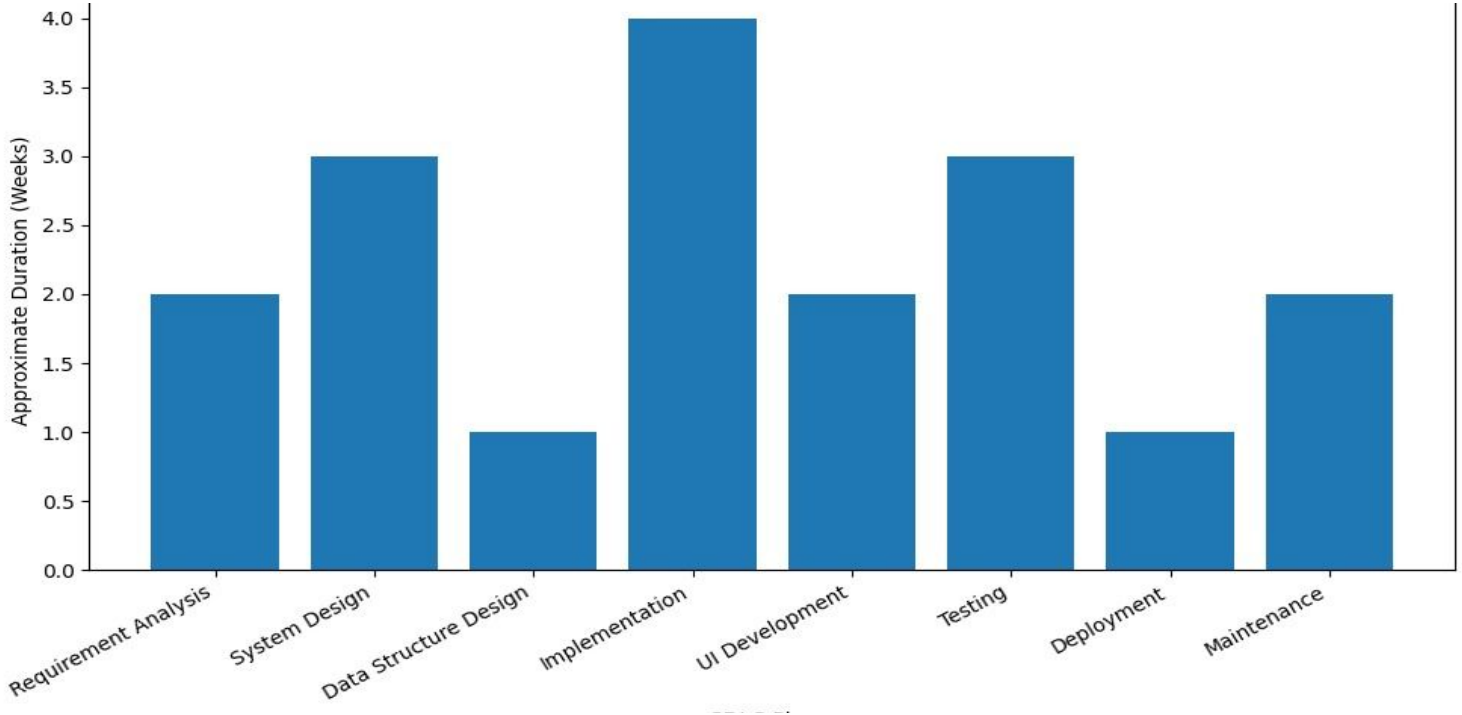

01
Requirement Analysis
Identify project inputs, outputs, functional requirements, and non-functional requirements for automated project planning and architecture generation.
02
System Design
Design the layered system architecture and define rule-based logic for planning schedules and architecture recommendations.
03
Data Structure Design
Design simple data structures to store project details and generated planning results.
04
User Interface Development
Implement a console-based or Flask web interface to collect user inputs and display outputs.
05
Testing
Test the system with different project inputs to verify correctnesss and reliability of generated plans and architectures.
06
Deployment
Deploy the application locally for demonstration and laboratory use.
Maintenance
Update rules and logic to improve accuracy and support future enhancements.
Waterfall
Model

GANTT CHART



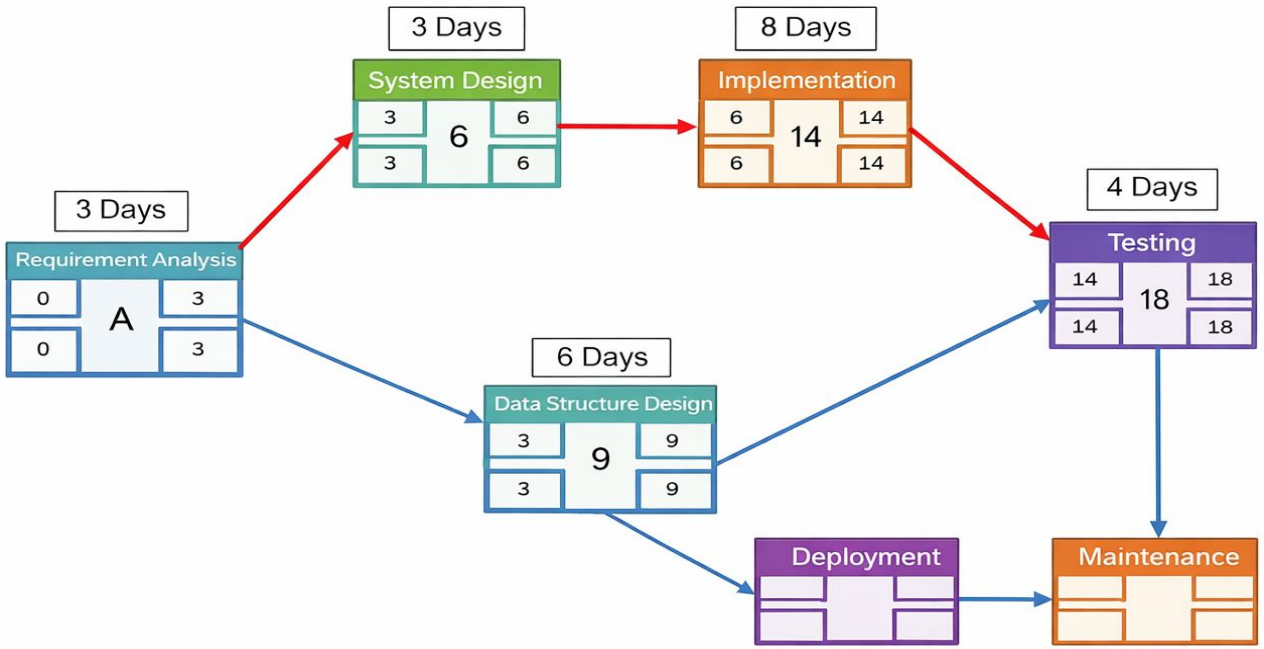
A Gantt chart is a project scheduling tool that graphically represents project activities against a time scale. In this project, the Gantt chart is generated for a three-month duration following the Waterfall model. The project timeline is divided using the 40–20–40 rule, where 40% of the time is allocated to planning and design, 20% to implementation, and the remaining 40% to testing, deployment, and maintenance.Accordingly, the initial month focuses on requirement analysis and system design, the middle phase emphasizes limited but focused implementation, and the final phase concentrates on testing, deployment, and documentation.

TIMELINE CHART



A timeline graph provides a high-level, phase-wise overview of the project schedule. For this project, the timeline spans three months starting from 6th January 2026.

ACTIVITY NETWORK CHART



An Activity Network Diagram represents the logical sequence and dependencies among project activities. In this project, the activity network follows a sequential Waterfall approach, where each phase begins only after the completion of the previous phase.The critical path includes requirement analysis, system design, database design, implementation, testing, deployment, and maintenance. Since the project follows the 40–20–40 rule, the implementation activity occupies a shorter duration compared to planning and testing phases. The activity network helps identify dependencies, visualize the critical path, and ensure timely completion of the three-month project.

RISK MANAGEMENT

Risk	Probability	Effect	Risk Type	Reducing Strategy	Potential Indicator of Risk
Latency	Mild	Mild	Product	Optimize backend code, database queries	Webpage loading time is very high
User Traffic	High	High	Product	Compress files (using gzip), optimize images, and minimize unnecessary code	The latency increases
Generation of Image	Low	Low	Product	You can use JavaScript within this attribute to change the src attribute to a valid default image URL	The image when page is loaded is not appearing
API Integration	Low	Low	Technical	Mismatched or expired API keys, tokens, or incorrect permissions are common issues	The Website is not working
Deployment CORS	Mild	Mild	Technical	allowing your frontend's domain, or use a proxy server	Blocks access to response Data
User Base	High	High	Business	Retaining existing customers is significantly more cost-effective than acquiring new ones	The Market is low and returns are less
Sectorial Adaption	Mild	Mild	Business	Effective adaptation often requires collaboration with different tools and technologies	Showing errors or unsatifactory results while using it for different sectors of Industry

Risk management, crucial for the Academic Project Planning and Automated Architecture Design Tool, identifies, analyzes, and mitigates risks to ensure successful completion within the three-month duration and maintain quality/schedule. Early identification of major risks is key. Using the 40–20–40 rule allocates more time to planning and testing, reducing implementation risk impact. Strategies like early requirement validation, modular design, and increased testing minimize uncertainties for smooth project execution.

TOOLS USED

WORK BREAKDOWN STRUCTURE: Napkin

WORK BREAKDOWN STRUCTURE(FLOWCHART): Mermaid

GANTT CHART: Mermaid

TIMELINE CHART: Excel

ACTIVITY NETWORK CHART: Canva

RISK MANAGEMENT CHART: Excel